CREATED AND MODIFIED CODE

Main.cpp

```
// ==============================================================
// STUDENT SUMMARY: Modified the boilerplate initialisation
// to manage the main window using smart pointers and adjusted
// the window properties.
// ==============================================================
#include "../JuceLibraryCode/JuceHeader.h"
#include "MainComponent.h"

class OtoDecksApplication  : public JUCEApplication
{
public:

    OtoDecksApplication() {}
    const String getApplicationName() override       { return
ProjectInfo::projectName; }
    const String getApplicationVersion() override    { return
ProjectInfo::versionString; }
    bool moreThanOneInstanceAllowed() override       { return true; }


    void initialise (const String& /*commandLine*/) override
    {
        // This method is where you should put your application's initialisation
code.
        // Creating the main window and setting the app name
        mainWindow.reset (new MainWindow (getApplicationName()));
    }

    void shutdown() override
    {

        // Cleaning up the main window to close the app properly
        mainWindow = nullptr; // (deletes our window)
    }


    void systemRequestedQuit() override
    {
        // This is called when the app is being asked to quit: you can ignore this
        // request and let the app carry on running, or call quit() to allow the
app to close.
        quit();
    }

    void anotherInstanceStarted (const String& /*commandLine*/) override
    {
```

```
        // When another instance of the app is launched while this one is running,
        // this method is invoked, and the commandLine parameter tells you what
        // the other instance's command-line arguments were.
    }


    /*
        This class implements the desktop window that contains an instance of
        our MainComponent class.
    */
    class MainWindow    : public DocumentWindow
    {
    public:
        MainWindow (String name)  : DocumentWindow (name,

Desktop::getInstance().getDefaultLookAndFeel()

.findColour (ResizableWindow::backgroundColourId),
                                                    DocumentWindow::allButtons)
        {
            setUsingNativeTitleBar (true);

            // Adding our MainComponent to the window
            setContentOwned (new MainComponent(), true);

           #if JUCE_IOS || JUCE_ANDROID
            setFullScreen (true);
           #else
            // Setting the window to be resizable and centring it on the screen
            setResizable (true, true);
            centreWithSize (getWidth(), getHeight());
           #endif

            // Making the window visible to the user
            setVisible (true);
        }

        void closeButtonPressed() override
        {
            // This is called when the user tries to close this window. Here,
we'll just
            // ask the app to quit when this happens, but you can change this to
do
            // whatever you need.
            JUCEApplication::getInstance()->systemRequestedQuit();
        }


    private:
        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainWindow)
    };

private:
    // Smart pointer to manage the main window's lifetime
```

```
    std::unique_ptr<MainWindow> mainWindow;
};

// This macro generates the main() routine that launches the app.
START_JUCE_APPLICATION (OtoDecksApplication)
```

DeckGUI.h

```
// ==========================================================
// STUDENT SUMMARY: Added declarations for custom data
// persistence functions (save/load cues and EQ) and new UI
// components (EQ sliders, BPM label, Hot Cues) without assistance.
// ==========================================================
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include "DJAudioPlayer.h"
#include "WaveformDisplay.h"

class DeckGUI    : public Component,
                   public Button::Listener,
                   public Slider::Listener,
                   public FileDragAndDropTarget,
                   public Timer
{
public:
    /** Constructor: sets up all the buttons, sliders, and labels, and starts the
UI timer.
     * @param player The audio player that this GUI will control.
     * @param formatManagerToUse The manager that handles different audio file
formats.
     * @param cacheToUse The cache used to store and display audio thumbnails.
     */
    DeckGUI(DJAudioPlayer* player,
           AudioFormatManager &    formatManagerToUse,
           AudioThumbnailCache &   cacheToUse );

    /** Destructor: stops the timer and tidies up when the deck is closed. */
    ~DeckGUI();

    /** Paints the background and standard visuals of the component.
     * @param g The graphics context used for drawing.
     */
    void paint (Graphics& g) override;

    /** Handles the responsive layout and positioning of all UI elements. */
    void resized() override;

    /** Loads audio file directly from deck or playlist.
     * @param audioURL The URL of the audio file to be loaded.
     */
```

```
    void loadFile(juce::URL audioURL);

    /** Sets the main theme colour for the deck's waveform.
     * @param c The colour to be applied to the waveform display.
     */
    void setMainColour(juce::Colour c);

private:
    /** Logic for when any button is clicked, like playing music or managing cues.
     * @param button A pointer to the button that was clicked.
     */
    void buttonClicked (Button * button) override;

    /** Updates the audio player and UI colours when sliders are moved.
     * @param slider A pointer to the slider that has changed value.
     */
    void sliderValueChanged (Slider *slider) override;

    /** Tells the system we are ready to receive files dragged onto the deck.
     * @param files A list of files being dragged over the component.
     * @return True if we are interested in the files, false otherwise.
     */
    bool isInterestedInFileDrag (const StringArray &files) override;

    /** Logic for loading a file when it's dropped onto the component.
     * @param files The list of files that were dropped.
     * @param x The x-coordinate of the drop location.
     * @param y The y-coordinate of the drop location.
     */
    void filesDropped (const StringArray &files, int x, int y) override;

    /** Regular callback to update the waveform playhead position. */
    void timerCallback() override;

    /** Saves current hot cue positions to a local file. */
    void saveCues();

    /** Loads saved hot cue positions for a specific track.
     * @param trackURL The URL of the track to load cues for.
     */
    void loadCues(juce::URL trackURL);

    /** Saves current EQ settings to a local file. */
    void saveEQ();

    juce::TextButton resetEQButton{"RESET EQ"};

    /** Loads saved EQ settings for a specific track.
     * @param trackURL The URL of the track to load EQ settings for.
     */
    void loadEQ(juce::URL trackURL);

    // UI Components
    juce::FileChooser fChooser{"Select a file..."};
```

```
    TextButton playButton{"PLAY"};
    TextButton stopButton{"STOP"};
    TextButton loadButton{"LOAD"};

    juce::TextButton cueButtons[8];
    juce::TextButton clearCuesButton{"Clear"};
    double cuePositions[8];

    juce::URL currentURL;

    Slider volSlider;
    Slider speedSlider;
    Slider posSlider;

    WaveformDisplay waveformDisplay;

    DJAudioPlayer* player;

    juce::Slider eqLowSlider;
    juce::Slider eqMidSlider;
    juce::Slider eqHighSlider;

    juce::Label volLabel{ "VOL", "VOL" };
    juce::Label speedLabel{ "SPD", "SPD" };
    juce::Label posLabel{ "POS", "POS" };
    juce::Label eqLabel{ "EQUALIZER", "EQUALIZER" };
    juce::Label lowLabel{ "LOW", "Low" }, midLabel{ "MID", "Mid" }, highLabel{
"HIGH", "High" };
    juce::Label bpmLabel;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (DeckGUI);
};
```

DeckGUI.cpp

```
// ==========================================================
// STUDENT SUMMARY: Implemented UI layout for new components,
// dynamic traffic-light colors for sliders, dynamic BPM
// calculation, and wrote data persistence methods (save/load)
// from scratch to fulfill Requirement R2 without assistance.
// ==========================================================
#include "../JuceLibraryCode/JuceHeader.h"
#include "DeckGUI.h"

// Constructor: sets up all the buttons, sliders, and labels, and starts the UI
timer
DeckGUI::DeckGUI(DJAudioPlayer* _player,
                 AudioFormatManager &    formatManagerToUse,
                 AudioThumbnailCache &    cacheToUse
           ) : player(_player),
```

```
                    waveformDisplay(formatManagerToUse, cacheToUse)
{
    // Adding the main playback and file buttons
    addAndMakeVisible(playButton);
    addAndMakeVisible(stopButton);
    addAndMakeVisible(loadButton);

    // Making the main deck sliders visible
    addAndMakeVisible(volSlider);
    addAndMakeVisible(speedSlider);
    addAndMakeVisible(posSlider);

    addAndMakeVisible(waveformDisplay);

    // Inside DeckGUI Constructor
    addAndMakeVisible(volLabel);
    addAndMakeVisible(speedLabel);
    addAndMakeVisible(posLabel);

    // Force text to the extreme left
    volLabel.setJustificationType(juce::Justification::centredLeft);
    speedLabel.setJustificationType(juce::Justification::centredLeft);
    posLabel.setJustificationType(juce::Justification::centredLeft);

    // Remove default border/internal gap
    volLabel.setBorderSize(juce::BorderSize<int>(5));
    speedLabel.setBorderSize(juce::BorderSize<int>(5));
    posLabel.setBorderSize(juce::BorderSize<int>(5));

    // Text from equalizer.
    addAndMakeVisible(eqLabel);
    eqLabel.setJustificationType(juce::Justification::centred);
    eqLabel.setFont(juce::Font(juce::FontOptions(12.0f).withStyle("Bold")));

    // Configuring the Reset EQ button
    addAndMakeVisible(resetEQButton);
    resetEQButton.addListener(this);

    // Clear Button.
    addAndMakeVisible(clearCuesButton);
    clearCuesButton.addListener(this);

    // EQ.
    addAndMakeVisible(lowLabel);
    addAndMakeVisible(midLabel);
    addAndMakeVisible(highLabel);

    lowLabel.setJustificationType(juce::Justification::centred);
    midLabel.setJustificationType(juce::Justification::centred);
    highLabel.setJustificationType(juce::Justification::centred);

    // Prepare 8 buttons Hot Cue
    for (int i = 0; i < 8; ++i) {
        cueButtons[i].setButtonText(juce::String(i + 1));
```

```
        addAndMakeVisible(cueButtons[i]);
        cueButtons[i].addListener(this);

        // Define initial time as -1.0 (empty button).
        cuePositions[i] = -1.0;
    }

    // Setting up the listeners for all our buttons and sliders
    playButton.addListener(this);
    stopButton.addListener(this);
    loadButton.addListener(this);

    volSlider.addListener(this);
    speedSlider.addListener(this);
    posSlider.addListener(this);

    volSlider.setRange(0.0, 1.0);
    speedSlider.setRange(0.0, 2.0);
    posSlider.setRange(0.0, 1.0);


    /* Set:
        - start volume in 0.50 (preventing user to not know if the program is
properly working).
        - Speed in 1.0, in order to start in regular speed.
        - Start in 0 seconds.
    */
    volSlider.setValue(0.5);
    speedSlider.setValue(1.0);
    posSlider.setValue(0.0);

    // Starting the timer so the UI keeps updating regularly
    startTimer(50);

    // Configurating sliders from EQ.
    eqLowSlider.setSliderStyle(juce::Slider::Rotary);
    eqLowSlider.setRange(0.1, 3.0);
    eqLowSlider.setValue(1.0);
    eqLowSlider.setTextBoxStyle(juce::Slider::NoTextBox, false, 0, 0);
    addAndMakeVisible(eqLowSlider);
    eqLowSlider.addListener(this);

    eqMidSlider.setSliderStyle(juce::Slider::Rotary);
    eqMidSlider.setRange(0.1, 3.0);
    eqMidSlider.setValue(1.0);
    eqMidSlider.setTextBoxStyle(juce::Slider::NoTextBox, false, 0, 0);
    addAndMakeVisible(eqMidSlider);
    eqMidSlider.addListener(this);

    eqHighSlider.setSliderStyle(juce::Slider::Rotary);
    eqHighSlider.setRange(0.1, 3.0);
    eqHighSlider.setValue(1.0);
    eqHighSlider.setTextBoxStyle(juce::Slider::NoTextBox, false, 0, 0);
    addAndMakeVisible(eqHighSlider);
```

```
    eqHighSlider.addListener(this);

    // Label BPM
    addAndMakeVisible(bpmLabel);
    bpmLabel.setText("BPM: --", juce::dontSendNotification);
    bpmLabel.setJustificationType(juce::Justification::centredLeft);
    bpmLabel.setColour(juce::Label::textColourId, juce::Colours::orange);

    // Adjust speedSlider.
    speedSlider.setRange(0.0, 2.0);
    speedSlider.setValue(1.0);


}

// The destructor: stops the timer and tidies up when the deck is closed
DeckGUI::~DeckGUI()
{
    stopTimer();
}

// Paints the background and standard visuals of the component
void DeckGUI::paint (Graphics& g)
{
   g.fillAll (getLookAndFeel().findColour (ResizableWindow::backgroundColourId));

    g.setColour (Colours::grey);
    g.drawRect (getLocalBounds(), 1);

    g.setColour (Colours::white);
    g.setFont (14.0f);
}

// Handles the responsive layout and positioning of all UI elements
void DeckGUI::resized()
{
    // Increased divisor to 15.0 to provide more vertical granularity
    double rowH = getHeight() / 15.0;
    int middleX = getWidth() / 2;
    int padding = 4;

    // Waveform
    waveformDisplay.setBounds(0, 0, getWidth(), (int)(rowH * 3.5));

    int startY = (int)(rowH * 3.5);
    int centralAreaH = (int)(rowH * 3.0);

    // Left Side: Play and Stop
    int bigButtonH = centralAreaH / 2;
    playButton.setBounds(juce::Rectangle<int>(0, startY, middleX,
bigButtonH).reduced(padding));
    stopButton.setBounds(juce::Rectangle<int>(0, startY + bigButtonH, middleX,
bigButtonH).reduced(padding));
```

```
    // Right Side: Hot Cues
    int cueW = middleX / 3;
    int cueH = centralAreaH / 3;

    for (int i = 0; i < 3; ++i)
        cueButtons[i].setBounds(juce::Rectangle<int>(middleX + (i * cueW), startY,
cueW, cueH).reduced(padding));

    for (int i = 0; i < 3; ++i)
        cueButtons[i + 3].setBounds(juce::Rectangle<int>(middleX + (i * cueW),
startY + cueH, cueW, cueH).reduced(padding));

    cueButtons[6].setBounds(juce::Rectangle<int>(middleX, startY + (cueH * 2),
cueW, cueH).reduced(padding));
    cueButtons[7].setBounds(juce::Rectangle<int>(middleX + cueW, startY + (cueH *
2), cueW, cueH).reduced(padding));
    clearCuesButton.setBounds(juce::Rectangle<int>(middleX + (cueW * 2), startY +
(cueH * 2), cueW, cueH).reduced(padding));

    // Main Sliders (VOL, SPD, POS)
    int sliderY = startY + centralAreaH;
    int sliderH = (int)rowH;
    int labelW = 40;
    int sliderW = getWidth() - labelW;

    volLabel.setBounds(0, sliderY, labelW, sliderH);
    volSlider.setBounds(labelW, sliderY, sliderW, sliderH);

    speedLabel.setBounds(0, sliderY + sliderH, labelW, sliderH);
    speedSlider.setBounds(labelW, sliderY + sliderH, sliderW, sliderH);

    posLabel.setBounds(0, sliderY + (sliderH * 2), labelW, sliderH);
    posSlider.setBounds(labelW, sliderY + (sliderH * 2), sliderW, sliderH);

    // Equalizer Section
    int eqLabelY = sliderY + (sliderH * 3);
    int eqLabelH = (int)(rowH * 0.6);

    // EQ Title
    eqLabel.setBounds(0, eqLabelY, getWidth(), eqLabelH);

    // EQ Knobs (The 3 sliders)
    int eqKnobsY = eqLabelY + eqLabelH;
    // Maintained the increased knob size (2.2) for better visibility
    int eqKnobsHeight = (int)(rowH * 2.2);
    int knobWidth = getWidth() / 3;

    eqLowSlider.setBounds(juce::Rectangle<int>(0, eqKnobsY, knobWidth,
eqKnobsHeight).reduced(padding));
    eqMidSlider.setBounds(juce::Rectangle<int>(knobWidth, eqKnobsY, knobWidth,
eqKnobsHeight).reduced(padding));
    eqHighSlider.setBounds(juce::Rectangle<int>(knobWidth * 2, eqKnobsY,
knobWidth, eqKnobsHeight).reduced(padding));
```

```
    // EQ Labels (Low, Mid, High) placed directly below the knobs
    int eqTextY = eqKnobsY + eqKnobsHeight - 5;
    // Height maintained to ensure "g" in "High" is not cut off
    int eqTextH = (int)(rowH * 0.6);

    lowLabel.setBounds(0, eqTextY, knobWidth, eqTextH);
    midLabel.setBounds(knobWidth, eqTextY, knobWidth, eqTextH);
    highLabel.setBounds(knobWidth * 2, eqTextY, knobWidth, eqTextH);

    // Reset EQ Button - repositioned with safety margin to avoid overlapping LOAD
button
    int resetBtnY = eqTextY + eqTextH + 5;
    int resetBtnHeight = (int)(rowH * 0.8);
    resetEQButton.setBounds(juce::Rectangle<int>(getWidth() / 4, resetBtnY,
getWidth() / 2, resetBtnHeight).reduced(padding));

    // Load Button - Anchored to the very bottom of the component
    int loadBtnHeight = (int)(rowH * 1.2);
    loadButton.setBounds(juce::Rectangle<int>(0, getHeight() - loadBtnHeight,
getWidth(), loadBtnHeight).reduced(padding));

    // BPM visualization at the top right
    bpmLabel.setBounds(getWidth() - 100, 5, 90, 20);
}

// Logic for when any button is clicked, like playing music or managing cues
void DeckGUI::buttonClicked(Button* button)
{
    // Detecting which button was clicked and starting the action
    if (button == &playButton)
    {
        player->start();
    }
    if (button == &stopButton)
    {
        player->stop();
    }
    if (button == &loadButton)
    {
        auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles;
        fChooser.launchAsync(fileChooserFlags, [this](const juce::FileChooser&
chooser)
        {
            juce::File chosenFile = chooser.getResult();
            if (chosenFile.exists()){
                juce::URL fileURL = juce::URL{chosenFile};
                loadFile(fileURL);
            }
        });
    }

    if (button == &resetEQButton)
        {
            // Reset the three sliders back to 1.0 (flat) and notify the listener
```

```
            eqLowSlider.setValue(1.0, juce::sendNotification);
            eqMidSlider.setValue(1.0, juce::sendNotification);
            eqHighSlider.setValue(1.0, juce::sendNotification);
        }

    for (int i = 0; i < 8; ++i)
    {
        if (button == &cueButtons[i])
        {
            // Set/Overwrite Cue point (Shift or Empty)
            if (cuePositions[i] == -1.0 ||
juce::ModifierKeys::getCurrentModifiers().isShiftDown())
            {
                cuePositions[i] = player->getPositionRelative();

                // When clicked.
                cueButtons[i].setColour(juce::TextButton::buttonColourId,
juce::Colours::grey);
                cueButtons[i].setColour(juce::TextButton::textColourOffId,
juce::Colours::black);

                saveCues();
            }
            else
            {
                // Jump to Cue point
                player->setPositionRelative(cuePositions[i]);
            }
        }
    }

    if (button == &clearCuesButton)
    {
        // Cleaning all the hot cues we've set
        for (int i = 0; i < 8; ++i)
        {
            cuePositions[i] = -1.0;

            // Reset to default look
            cueButtons[i].removeColour(juce::TextButton::buttonColourId);
            cueButtons[i].removeColour(juce::TextButton::textColourOffId);
        }
        // Save Cues.
        saveCues();
    }
}

// Updates the audio player and UI colours when sliders are moved
void DeckGUI::sliderValueChanged (Slider *slider)
{
    if (slider == &volSlider)
    {
       double val = slider->getValue();
        player->setGain(val);
```

```
        // Changing the slider colour based on the volume level
        if (val > 0.8) {
            slider->setColour(juce::Slider::thumbColourId, juce::Colours::red);
            slider->setColour(juce::Slider::trackColourId,
juce::Colours::red.withAlpha(0.5f));
        } else if (val > 0.5) {
            slider->setColour(juce::Slider::thumbColourId, juce::Colours::orange);
            slider->setColour(juce::Slider::trackColourId,
juce::Colours::orange.withAlpha(0.5f));
        } else {
            slider->setColour(juce::Slider::thumbColourId, juce::Colours::green);
            slider->setColour(juce::Slider::trackColourId,
juce::Colours::green.withAlpha(0.5f));
        }
    }

    if (slider == &speedSlider)
    {
        double speedVal = slider->getValue();
        player->setSpeed(speedVal);

        // Colours
        if (speedVal > 1.0) slider->setColour(juce::Slider::thumbColourId,
juce::Colours::red);
        else if (speedVal < 1.0) slider->setColour(juce::Slider::thumbColourId,
juce::Colours::orange);
        else slider->setColour(juce::Slider::thumbColourId, juce::Colours::green);

        // R5: Live BPM calculation
        double baseBpm = player->getBpm();
        double liveBpm = baseBpm * speedVal;

        bpmLabel.setText("BPM: " + juce::String(liveBpm, 1),
juce::dontSendNotification);
    }

    if (slider == &posSlider)
    {
        // Jumping to the specific track position
        player->setPositionRelative(slider->getValue());
    }

    if (slider == &eqLowSlider) {
        player->setEqLow((float)slider->getValue());
        saveEQ();
    }
    if (slider == &eqMidSlider) {
        player->setEqMid((float)slider->getValue());
        saveEQ();
    }
    if (slider == &eqHighSlider) {
        player->setEqHigh((float)slider->getValue());
        saveEQ();
```

```
    }

}

// Tells the system we are ready to receive files dragged onto the deck
bool DeckGUI::isInterestedInFileDrag (const StringArray& /*files*/)
{
  // We're always interested in new tracks!
  return true;
}

// Logic for loading a file when it's dropped onto the component
void DeckGUI::filesDropped (const StringArray& files, int /*x*/, int /*y*/)
{
  // If a single file is dropped, load it into the player
  if (files.size() == 1)
  {
    player->loadURL(URL{File{files[0]}});
  }
}

// Regular callback to update the waveform playhead position
void DeckGUI::timerCallback()
{
    // Moving the waveform playhead based on the audio position
    waveformDisplay.setPositionRelative(
            player->getPositionRelative());
}

// Coordinates loading a file into the player, waveform, cues, and EQ
void DeckGUI::loadFile(juce::URL audioURL)
{
    // Sends the file to the audio engine.
    player->loadURL(audioURL);

    // Send the file to draw the wave chart in the screen.
    waveformDisplay.loadURL(audioURL);

    loadCues(audioURL);

    loadEQ(audioURL);
}

// Saves current hot cue positions to a local file
void DeckGUI::saveCues()
{
    if (currentURL.isEmpty()) return;

    // Writing the cue points to a local file for the next time we load the track
    juce::String fileName = currentURL.getFileName() + ".cues";
    juce::File cueFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile(fi
leName);
```

```
    juce::FileOutputStream output(cueFile);
    if (!output.openedOk()) return;

    output.setPosition(0);
    output.truncate();

    for (int i = 0; i < 8; ++i) {
        output.writeText(juce::String(cuePositions[i]) + "\n", false, false,
nullptr);
    }
}

// Loads saved hot cue positions for a specific track
void DeckGUI::loadCues(juce::URL trackURL)
{
    currentURL = trackURL;

    // Resetting the UI for a fresh start with this track
    for (int i = 0; i < 8; ++i) {
        cuePositions[i] = -1.0;
        cueButtons[i].setButtonText(juce::String(i + 1));
    }

    juce::String fileName = currentURL.getFileName() + ".cues";
    juce::File cueFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile(fi
leName);

    if (cueFile.existsAsFile()) {
        juce::StringArray lines;
        cueFile.readLines(lines);

        for (int i = 0; i < 8 && i < lines.size(); ++i) {
            double pos = lines[i].getDoubleValue();
            cuePositions[i] = pos;

            if (pos != -1.0) {
                cueButtons[i].setButtonText(juce::String(i + 1)); // Without the
"ON"
                cueButtons[i].setColour(juce::TextButton::buttonColourId,
juce::Colours::grey);
                cueButtons[i].setColour(juce::TextButton::textColourOffId,
juce::Colours::black);
            }
        }
    }
}

// Saves current EQ settings to a local file
void DeckGUI::saveEQ()
{
    if (currentURL.isEmpty()) return;

    // Saving the EQ settings so the track sounds the same next time
```

```
    juce::String fileName = currentURL.getFileName() + ".eq";
    juce::File eqFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile(fi
leName);

    juce::FileOutputStream output(eqFile);
    if (!output.openedOk()) return;

    output.setPosition(0);
    output.truncate();

    // Save Low, Mid, High
    output.writeText(juce::String(eqLowSlider.getValue()) + "\n", false, false,
nullptr);
    output.writeText(juce::String(eqMidSlider.getValue()) + "\n", false, false,
nullptr);
    output.writeText(juce::String(eqHighSlider.getValue()) + "\n", false, false,
nullptr);
}

// Loads saved EQ settings for a specific track
void DeckGUI::loadEQ(juce::URL trackURL)
{
    currentURL = trackURL;

    // Searching for saved EQ settings in the user's documents
    juce::String fileName = currentURL.getFileName() + ".eq";
    juce::File eqFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile(fi
leName);

    if (eqFile.existsAsFile()) {
        juce::StringArray lines;
        eqFile.readLines(lines);

        if (lines.size() >= 3) {
            eqLowSlider.setValue(lines[0].getFloatValue(),
juce::sendNotification);
            eqMidSlider.setValue(lines[1].getFloatValue(),
juce::sendNotification);
            eqHighSlider.setValue(lines[2].getFloatValue(),
juce::sendNotification);
            return;
        }
    }

    // Setting defaults if no settings are found
    eqLowSlider.setValue(1.0, juce::sendNotification);
    eqMidSlider.setValue(1.0, juce::sendNotification);
    eqHighSlider.setValue(1.0, juce::sendNotification);
}

// Customises the waveform colour for this deck
void DeckGUI::setMainColour(juce::Colour c)
```

```
{
  waveformDisplay.setWaveformColour(c);
}
```

DJAudioPlayer.h

```
// ==============================================================
// STUDENT SUMMARY: Added declarations for DSP equalizer
// filters (ProcessorChain) and BPM metadata extraction logic.
// ==============================================================
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include <juce_dsp/juce_dsp.h>

class DJAudioPlayer : public AudioSource {
  public:

    /** Constructor: initializes the player and sets up the audio format manager.
     * @param _formatManager The manager that helps the player recognize different
audio files.
     */
    DJAudioPlayer(AudioFormatManager& _formatManager);

    /** Destructor: cleans up the resources when the player is no longer needed.
*/
    ~DJAudioPlayer();

    /** Prepares the audio source for playback.
     * @param samplesPerBlockExpected The number of samples the player should
process at a time.
     * @param sampleRate The sample rate of the audio output device.
     */
    void prepareToPlay (int samplesPerBlockExpected, double sampleRate) override;

    /** Fills the audio buffer with the next block of data to be played.
     * @param bufferToFill Information about the buffer that needs audio data.
     */
    void getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill) override;

    /** Releases audio resources when playback is finished or stopped. */
    void releaseResources() override;

    /** Loads a specific audio file from a URL.
     * @param audioURL The location of the audio file to load.
     */
    void loadURL(URL audioURL);

    /** Adjusts the volume gain of the player.
     * @param gain The volume level (usually between 0.0 and 1.0).
     */
```

```
    void setGain(double gain);

    /** Sets the playback speed ratio.
     * @param ratio The speed multiplier (e.g., 1.0 is normal, 2.0 is double
speed).
     */
    void setSpeed(double ratio);

    /** Moves the playhead to a specific time in seconds.
     * @param posInSecs The desired position in the track.
     */
    void setPosition(double posInSecs);

    /** Moves the playhead to a relative position (0.0 to 1.0).
     * @param pos The percentage of the track to jump to.
     */
    void setPositionRelative(double pos);

    /** Adjusts the low-frequency gain of the equaliser.
     * @param gainLinear The gain level for the low band.
     */
    void setEqLow(float gainLinear);

    /** Adjusts the mid-frequency gain of the equaliser.
     * @param gainLinear The gain level for the mid band.
     */
    void setEqMid(float gainLinear);

    /** Adjusts the high-frequency gain of the equaliser.
     * @param gainLinear The gain level for the high band.
     */
    void setEqHigh(float gainLinear);

    /** Starts the audio playback. */
    void start();

    /** Stops the audio playback. */
    void stop();

    /** get the relative position of the playhead */
    double getPositionRelative();

    /** Calculates the current BPM based on the track and playback speed.
     * @return The calculated beats per minute.
     */
    double getBpm();

private:
    AudioFormatManager& formatManager;
    std::unique_ptr<AudioFormatReaderSource> readerSource;
    AudioTransportSource transportSource;
    ResamplingAudioSource resampleSource{&transportSource, false, 2};
    double currentSampleRate = 44100.0;
```

```
    juce::dsp::ProcessorChain<juce::dsp::IIR::Filter<float>,
                              juce::dsp::IIR::Filter<float>,
                              juce::dsp::IIR::Filter<float>> eqChain;

    double currentBaseBpm = 120.0;
};
```

DJAudioPlayer.cpp

```
// ==============================================================
// STUDENT SUMMARY: Updated audio processing chain with IIR
// Filters (DSP), added null-pointer safety checks, and
// implemented EQ and BPM methods from scratch without assistance.
// ==============================================================
#include "DJAudioPlayer.h"

// The constructor: links the format manager to our player
DJAudioPlayer::DJAudioPlayer(AudioFormatManager& _formatManager)
: formatManager(_formatManager)
{

}

// The destructor: cleans up when the player is destroyed
DJAudioPlayer::~DJAudioPlayer()
{

}

// Cleanly release resources when they are no longer needed
void DJAudioPlayer::releaseResources()
{
    transportSource.releaseResources();
    resampleSource.releaseResources();
}

/*
Load an audio resource from a URL (e.g. file:///C:/Musics/track01.mp3) and starts
reproducing.

* This method is responsible for creating an audio format reader for the provided
resource,
* that manages a dynamic memory allocation via pointers (std::unique_ptr),
* and performing a transition of the audio stream to the transport source.

* audioURL The URL object pointing to the local or remote audio file.

* @note If the file format is unsupported or the resource is inaccessible,
* the function fails silently through a null pointer check,
* ensuring application stability.
*/
```

```
void DJAudioPlayer::loadURL(URL audioURL)
{
    // createReaderFor: reserve bytes in the Heap. Returns the position of the
first byte where is allocated in the memory (uses "new"), therefore creates in
heap.
    // createInputStream: If local file, opens in HD/SSD and prepare the OS to
start reading. The bool is to show or not the useProgressDialog.
    // Store the RAM address in reader.
    // The method returns a object juce::AudioFormatReader*, isntead of writting
it, Wrote auto* + name.
    // auto* reader has 8 bytes because it's only the address.
    // Variable reader is in stack, the object file is in heap.
    // NOTE: it's not the entire file, in the heap there's just a small space to
read the file little by little

    auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false));

    // If the variable reader points to somewhere, therefore there's a file.
    if (reader != nullptr) {

        // Search for BPM.
        juce::String bpmString = reader->metadataValues.getValue("bpm", "120");
        currentBaseBpm = bpmString.getDoubleValue();

        // If anything "weird", use 120.
        if (currentBaseBpm <= 0) currentBaseBpm = 120.0;

        // Create a variable called newSource.
        // unique_ptr creates a smart pointer (creates the address and when it's
not used anymore, deletes it).
        // It will store a AudioFormatReaderSource.
        // new AudioFormatReaderSource reserves a space in the heap, received the
address from reader.
        // This variable is created to play, stop, pause, etc.
        std::unique_ptr<AudioFormatReaderSource> newSource (new
AudioFormatReaderSource (reader, true));

        // transportSource is the objected created. The setSource will set from
where the
        // file comes from. Gets the newSource, starts from 0, don't use thread
manager (nullptr).
        transportSource.setSource (newSource.get(), 0, nullptr, reader-
>sampleRate);

        // Reset.
        readerSource.reset (newSource.release());
    } else {
        // Show an error if the file format is not recognised
        juce::AlertWindow::showMessageBoxAsync (
            juce::AlertWindow::WarningIcon,
            "Format Error",
            "The selected file is not supported.",
            "Ok"
```

```
        );
    }
}

// Adjust the volume, making sure it stays within a safe range
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0 || gain > 1.0)
    {
        std::cout << "DJAudioPlayer::setGain gain should be between 0 and 1" <<
std::endl;
    }
    else {
        transportSource.setGain(gain);
    }

}

// Adjust the playback speed (resampling ratio)
void DJAudioPlayer::setSpeed(double ratio)
{
  if (ratio < 0 || ratio > 2.00)
    {
        std::cout << "DJAudioPlayer::setSpeed ratio should be between 0 and 2" <<
std::endl;
    }
    else {
        resampleSource.setResamplingRatio(ratio);
    }
}

// Jump to a specific point in the song using seconds
void DJAudioPlayer::setPosition(double posInSecs)
{
    transportSource.setPosition(posInSecs);
}

// Jump to a specific point using a percentage (0.0 to 1.0)
void DJAudioPlayer::setPositionRelative(double pos)
{
     if (pos < 0 || pos > 1.0)
    {
        std::cout << "DJAudioPlayer::setPositionRelative pos should be between 0
and 1" << std::endl;
    }
    else {
        double posInSecs = transportSource.getLengthInSeconds() * pos;
        setPosition(posInSecs);
    }
}

// Start playing the music
void DJAudioPlayer::start()
{
```

```
    transportSource.start();
}

// Stop the music
void DJAudioPlayer::stop()
{
  transportSource.stop();
}

// Find out where we are in the song as a percentage
double DJAudioPlayer::getPositionRelative()
{
    return transportSource.getCurrentPosition() /
transportSource.getLengthInSeconds();
}

// Get the audio engine ready to play with the correct sample rate
void DJAudioPlayer::prepareToPlay (int samplesPerBlockExpected, double sampleRate)
{
    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    // Prepare DSP.
    currentSampleRate = sampleRate;
    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize = samplesPerBlockExpected;
    spec.numChannels = 2; // Stereo

    eqChain.prepare(spec);

    // Initialize filters (1 = flat / no changes).
    setEqLow(1.0f);
    setEqMid(1.0f);
    setEqHigh(1.0f);
}

// Process the audio through our resampling and EQ chain
void DJAudioPlayer::getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
{
    resampleSource.getNextAudioBlock(bufferToFill);

    juce::dsp::AudioBlock<float> block(*bufferToFill.buffer);

    auto subBlock = block.getSubsetChannelBlock(bufferToFill.startSample,
(size_t)bufferToFill.numSamples);

    juce::dsp::ProcessContextReplacing<float> context(subBlock);

    eqChain.process(context);
}

// Update the Low frequency shelf filter
void DJAudioPlayer::setEqLow(float gainLinear)
```

```
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makeLowShelf(currentSampleRate, 200.0f,
0.707f, gainLinear);
    *eqChain.get<0>().coefficients = *coeffs;
}

// Update the Mid frequency peak filter
void DJAudioPlayer::setEqMid(float gainLinear)
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makePeakFilter(currentSampleRate, 5000.0f,
0.707f, gainLinear);
    *eqChain.get<1>().coefficients = *coeffs;
}

// Update the High frequency shelf filter
void DJAudioPlayer::setEqHigh(float gainLinear)
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makeHighShelf(currentSampleRate, 4000.0f,
0.707f, gainLinear);
    *eqChain.get<2>().coefficients = *coeffs;
}

// Return the base BPM found in the track metadata
double DJAudioPlayer::getBpm()
{
    return currentBaseBpm;
}
```

MainComponent.h

```
// ==========================================================
// STUDENT SUMMARY: Included and instantiated the custom
// PlaylistComponent to integrate the music library.
// ==========================================================
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include "DJAudioPlayer.h"
#include "DeckGUI.h"
#include "PlaylistComponent.h"

class MainComponent   : public AudioAppComponent
{
public:
    /** Constructor: sets up the audio format manager, players, and the main
layout. */
    MainComponent();
```

```
    /** Destructor: shuts down the audio system and cleans up resources. */
    ~MainComponent();

    /** Prepares the audio channels and mixer for playback.
     * @param samplesPerBlockExpected The number of samples the audio device will
request.
     * @param sampleRate The output sample rate of the audio device.
     */
    void prepareToPlay (int samplesPerBlockExpected, double sampleRate) override;

    /** Combines the audio streams from both decks and sends them to the output.
     * @param bufferToFill Information about the audio buffer to be processed.
     */
    void getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill) override;

    /** Releases audio resources when they are no longer needed. */
    void releaseResources() override;

    /** Paints any background visuals or branding for the main application window.
     * @param g The graphics context used for drawing.
     */
    void paint (Graphics& g) override;

    /** Handles the responsive layout of the decks and the playlist. */
    void resized() override;

private:
    // Handles the different audio file formats
    AudioFormatManager formatManager;

    // Cache to store thumbnails so they don't have to be re-drawn constantly
    AudioThumbnailCache thumbCache{100};

    // Player and GUI for the first deck
    DJAudioPlayer player1{formatManager};
    DeckGUI deckGUI1{&player1, formatManager, thumbCache};

    // Player and GUI for the second deck
    DJAudioPlayer player2{formatManager};
    DeckGUI deckGUI2{&player2, formatManager, thumbCache};

    // Combines multiple audio sources into one output stream
    MixerAudioSource mixerSource;

    // The component that manages our track list and loading logic
    PlaylistComponent playlistComponent{&deckGUI1, &deckGUI2};

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainComponent)
};
```

MainComponent.cpp

```
// ==============================================================
// STUDENT SUMMARY: Set custom deck colors and redesigned
// the main layout (resized) to integrate and allocate space
// for the new PlaylistComponent.
// ==============================================================
#include "MainComponent.h"

MainComponent::MainComponent()
{
    // Make sure you set the size of the component after
    // you add any child components.
    setSize (1400, 900);

    // Colors from Decks.
    deckGUI1.setMainColour(juce::Colours::darkmagenta);
    deckGUI2.setMainColour(juce::Colours::cyan);

    // Some platforms require permissions to open input channels so request that
here
    if (RuntimePermissions::isRequired (RuntimePermissions::recordAudio)
        && ! RuntimePermissions::isGranted (RuntimePermissions::recordAudio))
    {
        RuntimePermissions::request (RuntimePermissions::recordAudio,
                                     [&] (bool granted) { if (granted)
setAudioChannels (2, 2); });
    }
    else
    {
        // Specify the number of input and output channels that we want to open
        setAudioChannels (0, 2);
    }

    // Making our GUI components visible on the screen
    addAndMakeVisible(deckGUI1);
    addAndMakeVisible(deckGUI2);
    addAndMakeVisible(playlistComponent);

    // Getting the format manager ready to handle common audio files
    formatManager.registerBasicFormats();
}

MainComponent::~MainComponent()
{
    // This shuts down the audio device and clears the audio source.
    shutdownAudio();
}

void MainComponent::prepareToPlay (int samplesPerBlockExpected, double sampleRate)
{
    // Getting both players ready for the audio stream
    player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
    player2.prepareToPlay(samplesPerBlockExpected, sampleRate);
```

```
    // Setting up the mixer and adding our decks as inputs
    mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);

 }
void MainComponent::getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
{
    // Fetching the combined audio from the mixer to play it out
    mixerSource.getNextAudioBlock(bufferToFill);
}

void MainComponent::releaseResources()
{
    // This will be called when the audio device stops, or when it is being
    // restarted due to a setting change.

    // For more details, see the help for AudioProcessor::releaseResources()
    player1.releaseResources();
    player2.releaseResources();
    mixerSource.releaseResources();
}

//==============================================================================
void MainComponent::paint (Graphics& g)
{
    // Component is opaque, so we must completely fill the background with a solid
colour.
    g.fillAll (getLookAndFeel().findColour (ResizableWindow::backgroundColourId));
}

void MainComponent::resized()
{
    // Divides screen.
    auto deckHeight = getHeight() * 0.6;

    // Decks upper part.
    deckGUI1.setBounds(0, 0, getWidth()/2, deckHeight);
    deckGUI2.setBounds(getWidth()/2, 0, getWidth()/2, deckHeight);

    // Playlist below decks.
   playlistComponent.setBounds(0, deckHeight, getWidth(), getHeight() -
deckHeight);
}
```

WaveformDisplay.h

```
// ==========================================================
// STUDENT SUMMARY: Added custom color variable and a method
// declaration to dynamically change waveform colors.
```

```
// ==============================================================
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"


class WaveformDisplay    : public Component,
                           public ChangeListener
{
public:
    /** Constructor: sets up the audio thumbnailer and registers the change
listener.
     * @param formatManagerToUse The manager that handles audio format reading.
     * @param cacheToUse The cache used to store and quickly retrieve thumbnails.
     */
    WaveformDisplay( AudioFormatManager &   formatManagerToUse,
                     AudioThumbnailCache &   cacheToUse );

    /** Destructor: cleans up the thumbnail resources when the display is
destroyed. */
    ~WaveformDisplay();

    /** Paints the visual representation of the audio waveform.
     * @param g The graphics context used for drawing the waveform and playhead.
     */
    void paint (Graphics& g) override;

    /** Handles the responsive layout of the waveform component. */
    void resized() override;

    /** Receives updates from the thumbnailer to redraw when the audio finishes
loading.
     * @param source The broadcaster that sent the change notification.
     */
    void changeListenerCallback (ChangeBroadcaster *source) override;

    /** Loads a specific audio file to be drawn as a waveform.
     * @param audioURL The URL of the audio file to process.
     */
    void loadURL(URL audioURL);

    /** set the relative position of the playhead*/
    void setPositionRelative(double pos);

    /** Changes the colour of the waveform lines for visual customisation.
     * @param newColour The new colour to be applied to the waveform.
     */
    void setWaveformColour(juce::Colour newColour);

private:
    // The core object that handles the audio visual data
    AudioThumbnail audioThumb;

    // Flag to check if we actually have a file to draw
```

```
    bool fileLoaded;

    // Current position of the playhead (0.0 to 1.0)
    double position;

    // The primary colour used to draw the waveform lines
    juce::Colour waveformColour = juce::Colours::orange;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (WaveformDisplay)
};
```

WaveformDisplay.cpp

```
// ==============================================================
// STUDENT SUMMARY: Updated the paint method to use dynamic
// coloring and implemented the color setter without assistance.
// ==============================================================
#include "../JuceLibraryCode/JuceHeader.h"
#include "WaveformDisplay.h"

WaveformDisplay::WaveformDisplay(AudioFormatManager &   formatManagerToUse,
                                 AudioThumbnailCache &  cacheToUse) :
                                 audioThumb(1000, formatManagerToUse, cacheToUse),
                                 fileLoaded(false),
                                 position(0)

{
  // Registering the display to listen for changes in the audio thumbnail
  audioThumb.addChangeListener(this);
}

WaveformDisplay::~WaveformDisplay()
{
}

void WaveformDisplay::paint (Graphics& g)
{
    g.fillAll (getLookAndFeel().findColour (ResizableWindow::backgroundColourId));
// clear the background

    g.setColour (Colours::grey);
    g.drawRect (getLocalBounds(), 1);   // draw an outline around the component

    g.setColour (waveformColour);

    if(fileLoaded)
    {
      // Drawing the actual waveform using the thumbnail data
      audioThumb.drawChannel(g,
        getLocalBounds(),
        0,
```

```
          audioThumb.getTotalLength(),
          0,
          1.0f
        );

        // Drawing the playhead indicator over the waveform
        g.setColour(Colours::lightgreen);
        g.drawRect(position * getWidth(), 0, getWidth() / 20, getHeight());
      }
      else
      {
        g.setFont (20.0f);
        g.drawText ("File not loaded...", getLocalBounds(),
                    Justification::centred, true);   // draw some placeholder text

      }
}

void WaveformDisplay::resized()
{
    // This method is where you should set the bounds of any child
    // components that your component contains..

}

void WaveformDisplay::loadURL(URL audioURL)
{
  // Clearing the previous thumbnail and loading the new audio source
  audioThumb.clear();
  fileLoaded  = audioThumb.setSource(new URLInputSource(audioURL));

  if (fileLoaded)
  {
    repaint();
  }
  else {
    std::cout << "Not loaded." << std::endl;
  }

}

void WaveformDisplay::changeListenerCallback (ChangeBroadcaster *source)
{
    // Redraw the component whenever the thumbnailer updates
    repaint();
}

void WaveformDisplay::setPositionRelative(double pos)
{
  // Only repaint if the position has actually changed to save resources
  if (pos != position)
  {
    position = pos;
    repaint();
```

```
  }
}

void WaveformDisplay::setWaveformColour(juce::Colour newColour)
{
    // Allowing the deck to customise its waveform colour
    waveformColour = newColour;
    repaint();
}
```

PlaylistComponent.h

```
// ============================================================
// STUDENT SUMMARY: This ENTIRE file was created by me from
// scratch without assistance to implement the Music Library
// (Requirement R2) and the unique Search feature.
// ============================================================

#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include <vector>
#include <string>

// Forward declaration: to compiler "Know" that the class exists.
class DeckGUI;

// Structure to store data from each music.
struct Track {
    juce::String title;
    juce::URL url;
    juce::String format;
    double lengthInSecs;
};

/**
     * Structure to represent a single audio track in the library.
     * It stores the file name (title) and its memory location (URL).
     * It has inheritance from juce Component, TableListBoxModel and
Button::Listener.
 */
class PlaylistComponent : public juce::Component,
                          public juce::TableListBoxModel,
                          public juce::Button::Listener,
                          public juce::TextEditor::Listener
{
public:
    PlaylistComponent(DeckGUI* deck1, DeckGUI* deck2);
    ~PlaylistComponent();

    /** Paints the background of the table rows. */
```

```
    void paint (juce::Graphics&) override;

     /** Draws the text/data into each cell of the library table. */
    void resized() override;

    /** Returns the total number of tracks currently in the library. */
    int getNumRows() override;

    /** Draws the background for each row in the table. */
    void paintRowBackground (juce::Graphics&, int rowNumber, int width, int
height, bool rowIsSelected) override;

    /** Draws the specific data (text) into each cell of the table. */
    void paintCell (juce::Graphics&, int rowNumber, int columnId, int width, int
height, bool rowIsSelected) override;

    /** Insert the interactive components (buttons) in the cells. */
    juce::Component* refreshComponentForCell (int rowNumber, int columnId, bool
isRowSelected, juce::Component* existingComponentToUpdate) override;

    /** Handles button clicks, specifically the 'Import' action for R2A. */
    void buttonClicked(juce::Button* button) override;

    /** Stores the actual state from vector of tracks in disk. */
    void saveLibrary();

    /** Reads the disk and build the vector of tracks before initialization. */
    void loadLibrary();

    /** XXX */
    juce::TextEditor searchInput;

    /** XXX */
    std::vector<Track> allTracks;

    /** */
    void textEditorTextChanged(juce::TextEditor& editor) override;

private:
    juce::TableListBox tableComponent;
    juce::TextButton importButton{ "IMPORT TO LIBRARY" };

    // Vector that stores multiple Track objects for the library.
    std::vector<Track> tracks;

    std::unique_ptr<juce::FileChooser> fChooser;

    // Pointers to decks.
    DeckGUI* deck1;
    DeckGUI* deck2;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (PlaylistComponent)
};
```

PlaylistComponent.cpp

```
// ============================================================
// STUDENT SUMMARY: This ENTIRE file was created by me from
// scratch without assistance to implement the Music Library
// (Requirement R2) and the unique Search feature.
// ============================================================
#include "PlaylistComponent.h"
#include "DeckGUI.h"

PlaylistComponent::PlaylistComponent(DeckGUI* _deck1, DeckGUI* _deck2):
deck1(_deck1), deck2(_deck2)
{
    // Configuration of columns from table.
    tableComponent.getHeader().addColumn("Track Title", 1, 300);
    tableComponent.getHeader().addColumn("Format", 2, 100);
    tableComponent.getHeader().addColumn("Duration", 3, 100);
    tableComponent.getHeader().addColumn("Load D1", 4, 80);
    tableComponent.getHeader().addColumn("Load D2", 5, 80);
    tableComponent.getHeader().addColumn("Remove", 6, 80);

    tableComponent.setModel(this);

    // Making the table and import button visible to the user
    addAndMakeVisible(tableComponent);
    addAndMakeVisible(importButton);

    // Setting up the listener for our import button
    importButton.addListener(this);

    // Adding the search bar and setting its placeholder text
    addAndMakeVisible(searchInput);
    searchInput.addListener(this);
    searchInput.setTextToShowWhenEmpty("Search tracks...",
juce::Colours::lightgrey);

    // Read saved file when opening the app.
    loadLibrary();
}

PlaylistComponent::~PlaylistComponent() {}

void PlaylistComponent::paint(juce::Graphics& g)
{
    // Filling the background with the default window colour

g.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundColourId));
}

void PlaylistComponent::resized()
{
```

```
    int toolBarHeight = 40;
    int padding = 5;

    // Positioning the top buttons and search input
    importButton.setBounds(padding, padding, (getWidth() / 4) - (padding * 2),
toolBarHeight - (padding * 2));
    searchInput.setBounds(getWidth() / 4, padding, (getWidth() / 4 * 3) - padding,
toolBarHeight - (padding * 2));

    // The table takes up the rest of the window space below the toolbar
    tableComponent.setBounds(0, toolBarHeight, getWidth(), getHeight() -
toolBarHeight);
}

// Returns the size of the vector (how many rows the table should draw).
int PlaylistComponent::getNumRows()
{
    return static_cast<int>(tracks.size());
}

// Draw the background of the lines (blue if selected, grey if not).
void PlaylistComponent::paintRowBackground(juce::Graphics& g, int rowNumber, int
width, int height, bool rowIsSelected)
{
    if (rowIsSelected) g.fillAll(juce::Colours::lightblue);
    else g.fillAll(juce::Colours::darkgrey);
}

// Draw the text in each cell based on the columnId.
void PlaylistComponent::paintCell(juce::Graphics& g, int rowNumber, int columnId,
int width, int height, bool rowIsSelected)
{
    if (rowNumber < getNumRows())
    {
        g.setColour(juce::Colours::white);

        if (columnId == 1) // Title.
            g.drawText(tracks[rowNumber].title, 2, 0, width - 4, height,
juce::Justification::centredLeft, true);

        if (columnId == 2) // Format.
            g.drawText(tracks[rowNumber].format, 2, 0, width - 4, height,
juce::Justification::centredLeft, true);

        if (columnId == 3) // Duration.
            g.drawText(juce::String( (int)tracks[rowNumber].lengthInSecs ) + "s",
2, 0, width - 4, height, juce::Justification::centredLeft, true);
    }
}

void PlaylistComponent::buttonClicked(juce::Button* button)
{
    if (button == &importButton)
    {
```

```
        // Define to open files and allow multiple files.
        auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles |

juce::FileBrowserComponent::canSelectMultipleItems;

        // Create the selector (using std::make_unique to manage memory).
        fChooser = std::make_unique<juce::FileChooser>("Select files to add to
library...",
                                                       juce::File{},
                                                       "*.mp3;*.wav;*.aif");

        // Assync form (modern JUCE).
        fChooser->launchAsync(fileChooserFlags, [this](const juce::FileChooser&
chooser)
        {
            auto results = chooser.getResults(); // Gets the list of selected
files

            for (const auto& file : results)
            {
                Track newTrack;
                newTrack.title = file.getFileName();
                newTrack.url = juce::URL{file};
                newTrack.format = file.getFileExtension();

                juce::AudioFormatManager formatManager;
                formatManager.registerBasicFormats();
                std::unique_ptr<juce::AudioFormatReader>
reader(formatManager.createReaderFor(file));

                if (reader != nullptr) {
                    newTrack.lengthInSecs = reader->lengthInSamples / reader-
>sampleRate;
                } else {
                    newTrack.lengthInSecs = 0;
                }

                tracks.push_back(newTrack);
            }

            // Music.
            allTracks = tracks;

            // Tells table to update the list in the screen.
            tableComponent.updateContent();

            // Persisting the updated track list to a file
            saveLibrary();
        });
    }
}

juce::Component* PlaylistComponent::refreshComponentForCell(int rowNumber, int
columnId, bool isRowSelected, juce::Component* existingComponentToUpdate)
```

```
{

    if (columnId == 4 || columnId == 5 || columnId == 6)
    {
        auto* btn = static_cast<juce::TextButton*>(existingComponentToUpdate);

        if (btn == nullptr)
        {
            // Defines the name for the button (columns 4, 5 and 6).
            juce::String btnName = (columnId == 4) ? "Deck 1" : (columnId == 5 ?
"Deck 2" : "Remove");
            btn = new juce::TextButton(btnName);
        }

        // Stores the actual row in the ID of the button for later reference.
        btn->setComponentID(juce::String(rowNumber));

        // Defines the action of the click in memory.
                btn->onClick = [this, btn, columnId]()
                {
                    int row = btn->getComponentID().getIntValue();

                    if (columnId == 4) {
                        // Send URL from music to deck 1.
                        deck1->loadFile(tracks[row].url);
                    } else if (columnId == 5) {
                        // Send URL from music to deck 2.
                        deck2->loadFile(tracks[row].url);

                    } else if (columnId == 6) {

                        // Using async call to safely remove tracks without
crashing the UI
                        juce::MessageManager::callAsync([this, row]() {
                            if (row < tracks.size()) {

                            // Deletes backup.
                            for (int i = 0; i < allTracks.size(); ++i) {
                                if (allTracks[i].url == tracks[row].url) {
                                    allTracks.erase(allTracks.begin() + i);
                                    break;
                                }
                            }
                            // Delete from screen.
                            tracks.erase(tracks.begin() + row);
                            tableComponent.updateContent();
                            saveLibrary();
                            }
                        });
                    }
                };
        return btn;
    }
    return nullptr;
```

```
}

void PlaylistComponent::saveLibrary()
{
    // Creates (or locates) a file "OtoDecksLibrary.txt" in the documents
directory of the OS.
    juce::File libraryFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile("O
toDecksLibrary.txt");

    // Opens the output flow of data.
    juce::FileOutputStream output(libraryFile);
    if (!output.openedOk()) return;

    output.setPosition(0);
    output.truncate();

    // Iterates through the vector and stores the path of each track.
    for (const auto& track : tracks)
    {
        // Collects the absolute path from Windows / Mac and adds a line break.
        juce::String filePath = track.url.getLocalFile().getFullPathName();
        output.writeText(filePath + "\n", false, false, nullptr);
    }
}

void PlaylistComponent::loadLibrary()
{
    // Points to the same persistence file used for saving.
    juce::File libraryFile =
juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile("O
toDecksLibrary.txt");

    if (libraryFile.existsAsFile())
    {
        // Read all lines from file once.
        juce::StringArray lines;
        libraryFile.readLines(lines);

        // Build the Track objects and allocate them in memory.
        for (juce::String line : lines)
        {
            if (line.isNotEmpty())
            {
                juce::File file{line};

                if (file.exists()) {
                    Track newTrack;
                    newTrack.title = file.getFileName();
                    newTrack.url = juce::URL{file};
                    newTrack.format = file.getFileExtension();

                    juce::AudioFormatManager formatManager;
                    formatManager.registerBasicFormats();
```

```
                    std::unique_ptr<juce::AudioFormatReader>
reader(formatManager.createReaderFor(file));

                    if (reader != nullptr) {
                        newTrack.lengthInSecs = reader->lengthInSamples / reader-
>sampleRate;
                    } else {
                        newTrack.lengthInSecs = 0;
                    }

                    tracks.push_back(newTrack);
                }
            }
        }

        // Music.
        allTracks = tracks;

        // Updates table with new loaded data.
        tableComponent.updateContent();
    }
}

void PlaylistComponent::textEditorTextChanged(juce::TextEditor& editor)
{
    if (searchInput.getText().isEmpty()) {
        // Restores everything if the text is erased.
        tracks = allTracks;
    } else {
        tracks.clear();
        for (const auto& track : allTracks) {
            // Compares the track title with the search input.
            if (track.title.containsIgnoreCase(searchInput.getText())) {
                tracks.push_back(track);
            }
        }
    }
    // Refreshing the table content to show filtered results
    tableComponent.updateContent();
}
```